

CSCI 432 Handout 09: Analyzing an Algorithm

Name: CLASS

5 March 2026

What algorithm are you analyzing today?

Answer

Closest Pair

1 What?

When analyzing algorithms, we first ask WHAT? That is, what is the problem that we want solved? Typically, in this description, we need to understand both what are the inputs *and* what are the outputs of the algorithm. When describing WHAT, it should be independent of HOW.

So, describe the WHAT for the algorithm you wish to analyze today.

What is the input?

Answer $Q = \text{an array of points in } \mathbb{R}^2$

What is the output? a pair of points from Q , $\langle p, q \rangle$

Answer

Use the previous two answers to concisely describe what is the problem? This often requires explaining the output in terms of the input.

Answer $\langle p, q \rangle \in Q^2$ has the property that no other pair $\langle p', q' \rangle \in Q^2$ with $p' \neq q'$ are closer than $\langle p, q \rangle$

$$\langle p, q \rangle = \operatorname{argmin}_{\langle p', q' \rangle \in Q^2 \setminus D} \|p' - q'\|_2, \text{ where } D = \{(x, x) \mid x \in \mathbb{R}\}$$

2 How?

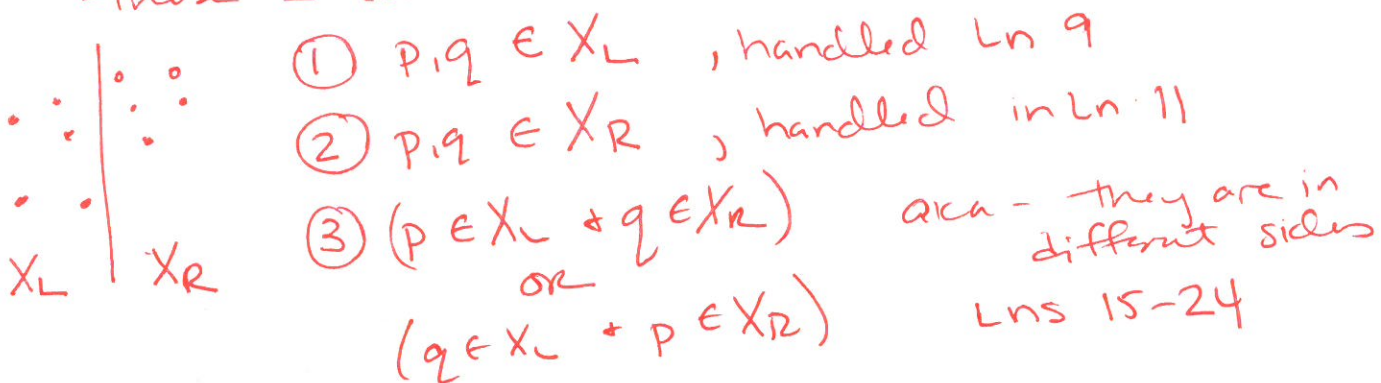
Often, it helps to walk through small examples. If you came up with the algorithm, this is a nice sanity check. If you are studying an algorithm given to you, this is helpful to better understand the algorithm. Do that now.

Answer try a Big example + believe in the recursion fairy!

If you are presenting an algorithm, you should describe how it works in pseudocode. More importantly, explain what is going on in the pseudocode in the prose text. The prose should point to the algorithm. (Remember, in LaTeX, algorithms are floating environments, so we need to be reminded to look at them). If you are working on analyzing an algorithm that is given to you in pseudocode, briefly explain HOW it works here. For now ignore base cases and focus on, *what is the key step?*

Answer

The closest pair (p, q) ~~could have~~ fits into one of these 3 cases:



$\{p, q\} \in \mathcal{Q}$

CASE 1: $p \in X_L$ $\leftarrow$ Case 1a: $q \in X_L$
 Case 1b: $q \notin X_L \Rightarrow q \in X_R$

CASE 2: $p \notin X_L$ $\leftarrow$ Case 2a: $q \in X_L$
 Case 2b: $q \notin X_L \Rightarrow q \in X_R$
 $\Rightarrow p \in X_R$

3 How Fast?

Option A: What is the runtime? If this is a recursive algorithm, you are expected to explicitly state the recurrence relation. *for Closest Pair Helper*

Answer

$$\begin{aligned} T(n) &= \Theta(1) + 2T(n/2) + \Theta(n) + \Theta(n) + \Theta(1) \\ &= 2T(n/2) + \Theta(n) \end{aligned}$$

same recurrence rel. as merge sort!
know: $T(n) \in \Theta(n \log n)$

Option B: Sometimes, it is hard to pin down the runtime, yet we still want to ensure that the algorithm terminates. For this, we use decrementing functions to prove that loops/recursions terminate. For the next few questions, try to use a decrementing function to prove termination.

1. What is the decrementing function for this algorithm?
Try it!

2. Justify (=prove) why function is well-defined. That is, why is the output of the function a natural number?
Try it!

3. Justify (=prove) why this decrements each time it reaches the top of the loop (or each time it goes into a new recursive call).
Try it!

4 Why? The Loop Invariant

Finally, we prove why the algorithm works. Typically, this is done with a loop (or recursion) invariant. Here, we focus on setting up the loop invariant. What are the following statements (start with your best guess, then come back and revise if needed):

The loop guard G :

The post-condition Q :

The pre-condition P :

The loop invariant $L = L_i$:

In §23.1, the pseudocode would not have made sense without a few paragraphs of motivation beforehand. By contrast, in this section we will give the pseudocode first; the proof of correctness will elucidate some of the strange-seeming choices that we make in the algorithm. This is more or less how the algorithm is presented in §33.4 of CLRS.

The algorithm begins by pre-sorting the points in Q according to their x - and y -coordinates:

Algorithm: CLOSEST-PAIR(Q)

```

1  $X \leftarrow$  the points of  $Q$ , sorted by  $x$ -coordinate
2  $Y \leftarrow$  the points of  $Q$ , sorted by  $y$ -coordinate
3 return CLOSEST-PAIR-HELPER( $X, Y$ )

```

Most of the work is done by the helper function CLOSEST-PAIR-HELPER, which makes recursive calls to itself:

Algorithm: CLOSEST-PAIR-HELPER(X, Y)

```

1 if  $|X| \leq 3$  then
2   Solve the problem by brute force and return
3  $x^* \leftarrow$  the median  $x$ -coordinate of  $X$ 
4 Let  $X_L \subseteq X$  consist of those points with  $x$ -coordinate  $\leq x^*$ 
5 Let  $X_R \subseteq X$  consist of those points with  $x$ -coordinate  $> x^*$ 
6 Let  $Y_L \subseteq Y$  consist of those points which are in  $X_L$ 
7 Let  $Y_R \subseteq Y$  consist of those points which are in  $X_R$ 
8  $\triangleright$  Find the closest two points in the left half
9  $\langle p_L, q_L \rangle \leftarrow$  CLOSEST-PAIR-HELPER( $X_L, Y_L$ )
10  $\triangleright$  Find the closest two points in the right half
11  $\langle p_R, q_R \rangle \leftarrow$  CLOSEST-PAIR-HELPER( $X_R, Y_R$ )
12  $\delta_L \leftarrow$  distance from  $p_L$  to  $q_L$ 
13  $\delta_R \leftarrow$  distance from  $p_R$  to  $q_R$ 
14  $\delta \leftarrow \min\{\delta_L, \delta_R\}$ 
15  $Y' \leftarrow$  those points in  $Y$  whose  $x$ -coordinate is within  $\delta$  of  $x^*$ 
16  $\triangleright$  Recall that  $Y'$  is already sorted by  $y$ -coordinate
17  $\epsilon \leftarrow \infty$ 
18 for  $i \leftarrow 1$  to  $|Y'| - 1$  do
19    $p \leftarrow Y'[i]$ 
20   for  $q$  in  $Y'[i+1, \dots, \min\{i+7, |Y'|\}]$  do
21     if  $\epsilon >$  distance from  $p$  to  $q$  then
22        $\epsilon \leftarrow$  distance from  $p$  to  $q$ 
23        $p^* \leftarrow p$ 
24        $q^* \leftarrow q$ 
25 if  $\epsilon < \delta$  then
26   return  $\langle p^*, q^* \rangle$ 
27 else if  $\delta_R < \delta_L$  then
28   return  $\langle p_R, q_R \rangle$ 
29 else
30   return  $\langle p_L, q_L \rangle$ 

```

(note: typos in Lines 5, 12)

median = $\frac{1}{2}$ elts "above" + $\frac{1}{2}$ elts "below"

$\Theta(1)$

$\Theta(1)$ assuming just indexing x^*

$T(n/2)$

$T(n/2)$

$\Theta(1)$

$\Theta(n)$

$\Theta(n-1) = \Theta(n)$

$\Theta(1)$

Δ means "comment"

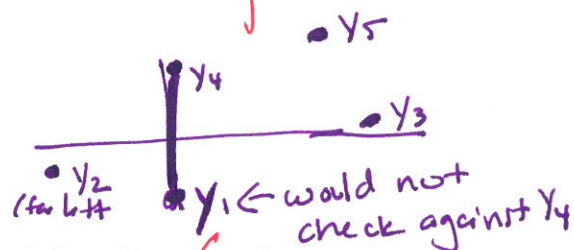
Closest Pair:

$$|Q| = n$$

Q1: What would the brute force sol'n look like?

Check every pair, of which there are $\binom{n}{2} = \Theta(n^2)$
const. work per pair to keep track of
the current closest pair among all I've seen so far
 $\Rightarrow \Theta(n^2 \cdot 1) = \Theta(n^2)$

Q2: On the 7 in Line 20 ... what if we used $i+2$ instead of $i+7$? Would we miss something?
Can you find an example?



Q3: Example where Y' has only 2 pts (or a constant # of them)?



Q4 Example where Y' has all the pts (or $\Theta(n)$ of them)?



Next, we use this loop invariant (and other statements above) to prove partial correctness. There are three parts to partial correctness: Initialization, Maintenance, and End.

Initialization: If the preconditions are met and if we enter the loop, then the loop invariant is correct. $(P \wedge G \implies L)$.

Answer

Maintenance: If we entered the loop and the loop invariant held, then when we exit the loop, the loop invariant will still hold: $(G \wedge L_i \implies L_{i+1})$.

Answer

End: If the loop ends and the loop invariant was correct, then the algorithm is correct. $(L \wedge \neg G \implies Q)$.

Answer